2026–2027

جب کوئی قوم فن اور علم
سے عاری ہو جاتی ہے
تو وہ غربت کو دعوت
دیتی ہے اور جب غربت
آتی ہے تو وہ ہزاروں
جرائم کو جنم دیتی ہے۔

"When a nation becomes devoid of art and learning, it invites poverty and when poverty comes it brings in its wake thousands of crimes."

– Sir Syed Ahmad Khan

B.Sc. (CA) VII Semester Lab Manual

Lab-VII

Course Code- CABSMJ7P07

DEPARTMENT OF COMPUTER SCIENCE

ALIGARH MUSLIM UNIVERSITY, ALIGARH

2026–2027

CREDITS

Lab Manual Up-gradation Committee

The following lab manual up-gradation committee updated the lab manual:

- Prof. Arman Rasool Faridi (Chairperson)
- Prof. Mohammad UbaidUllah Bokhari
- Prof. Aasim Zafar
- Prof. Suhel Mustajab
- Dr. Shafiqul Abidin
- Dr. Asif Irshad Khan
- Dr. Faisal Anwar
- Dr. Mohammad Sajid
- Dr. Mohammad Nadeem
- Dr. Faraz Masood

Original Design Committee

The following committee members originally designed the lab manual:

- Prof. Mohammad Ubaidullah Bokhari
- Dr. Arman Rasool Faridi
- Dr. Faisal Anwer
- Prof. Aasim Zafar (Convener)

Design & Compilation

Dr. Faraz Masood

Revised Edition: July 2026

Department of Computer Science, A.M.U., Aligarh (U.P.), India

LAB MANUAL: LAB VII

B.Sc. (CA) VII Semester | CABSMJ7P01

CONTENTS

Contents

COURSE TITLE:	Lab-VII	**COURSE CODE:**	CABSMJ7P01
CREDIT:	4	**PERIODS PER WEEK:**	6
CONTINUOUS ASSESSMENT:	60	**EXAMS:**	40

COURSE DESCRIPTION

This course provides students with a practical and conceptual understanding of Artificial Intelligence (AI) and Machine Learning (ML) through hands-on lab sessions. Students will explore fundamental AI concepts such as search algorithms, informed heuristics, and problem-solving techniques, as well as core ML methodologies, including supervised and unsupervised learning, data preprocessing, and performance evaluation. The course introduces essential programming constructs in Python, covering advanced data structures, functional programming, and object-oriented design, laying a foundation for structured model development. Students will work with real-world datasets to practice data cleaning, transformation, and integration, and will implement algorithms such as K-Nearest Neighbors (KNN), Decision Trees, Logistic Regression, Artificial Neural Networks (ANNs), and clustering techniques. Additionally, students will gain exposure to critical machine learning concepts like underfitting, overfitting, cross-validation, and performance metrics, enabling them to build, evaluate, and optimize predictive models. By the end of the course, students will have developed both the theoretical understanding and practical skills required to apply AI and ML techniques to real-world problems.

CONTENT

The lab manual provides a structured 14-week program covering both AI and ML concepts. Early sessions focus on advanced Python programming, functional programming constructs, and object-oriented principles. Students then transition into AI-specific topics, including search algorithms (BFS, DFS, A*, and Best-First Search) and problem-solving strategies. Subsequent sessions emphasize ML workflows, beginning with data preprocessing (cleaning, integration, transformation, and feature selection) and extending to supervised and unsupervised learning methods. Students will work with KNN, Linear Regression, Decision Trees, Logistic Regression, and ANN for classification and regression tasks. They will also explore clustering techniques (K-Means and Hierarchical Clustering) and understand key issues like underfitting, overfitting, and data separability. The final sessions focus on model evaluation, where students learn to interpret confusion matrices, accuracy, precision, recall, and various error measures (MAE, RMSE, R^2). Each session integrates theoretical understanding with practical, Python-based implementation, using real and simulated datasets.

OBJECTIVES

This course aims to:

- **Develop Proficiency in Python for AI/ML Applications:** Equip students with advanced programming skills, including functional programming, data structure manipulation, and object-oriented modeling, to support AI and ML implementations.
- **Understand and Apply Core AI Concepts:** Introduce students to AI problem-solving techniques, including state-space search, heuristic-based informed search, and pathfinding algorithms, enabling them to solve classical AI problems.
- **Gain Expertise in Data Preprocessing and Feature Engineering:** Train students to clean, transform, and integrate real-world datasets, applying techniques like normalization, standardization, discretization, and feature selection to prepare data for machine learning models.
- **Master Supervised and Unsupervised Learning Algorithms:** Provide hands-on experience with KNN, Decision Trees, Logistic Regression, Neural Networks, K-Means, and Hierarchical Clustering, focusing on model training, prediction, and evaluation.
- **Understand Model Evaluation and Optimization:** Teach students to identify underfitting and overfitting, apply cross-validation, and interpret evaluation metrics such as accuracy, confusion matrix, precision, recall, MAE, RMSE, and R^2 to improve model performance.
- **Bridge Theory and Real-World Application:** Encourage the practical application of AI/ML concepts by using real datasets, enabling students to solve real-world predictive and analytical problems effectively.

OUTCOMES

After completing this course, the students would be able to:

- **Write Efficient Python Code for AI/ML Tasks:** Utilize advanced programming techniques, including functional programming and OOP, to develop modular and reusable code for AI/ML applications.
- **Implement Classical AI Algorithms:** Design and implement BFS, DFS, A*, and Best-First Search to solve search and pathfinding problems, demonstrating an understanding of heuristic-based optimization.
- **Prepare and Transform Real-World Data for ML:** Perform comprehensive data preprocessing, including cleaning, handling missing values, outlier detection, normalization, standardization, and feature engineering, ensuring data readiness for modeling.
- **Build and Evaluate Machine Learning Models:** Train, test, and evaluate supervised and unsupervised models using KNN, Decision Trees, Logistic Regression, Neural Networks, K-Means, and Hierarchical Clustering, and interpret their results effectively.
- **Analyze Model Performance Using Advanced Metrics:** Apply accuracy, confusion matrices, precision, recall, MAE, RMSE, and R^2 to assess model performance and make informed improvements.
- **Identify and Address Model Optimization Challenges:** Detect and mitigate underfitting and overfitting issues, choose appropriate cross-validation techniques, and select models based on data separability and complexity.
- **Apply AI/ML Techniques to Real-World Problems:** Solve real-world predictive modeling, classification, and clustering problems, demonstrating both theoretical knowledge and practical implementation skills.

RULES AND REGULATIONS

Students are required to strictly adhere to the following rules.

- The students must complete the weekly activities/assignments well in time (i.e., within the same week).
- The students must maintain the Lab File of their completed problems in the prescribed format (Appendix-1).
- The students must get the completed weekly problems checked and signed by the concerned teachers in the Lab in the immediate succeeding week. Failing which the activities/assignments for that week will be treated as incomplete.
- At least TEN (10) such timely completed and duly signed weekly activities/assignments are compulsory, failing which students will not be allowed to appear in the final Lab Examination.
- Late submission would not be accepted after the due date.

- Cooperate, collaborate and explore for the best individual learning outcomes but copying is strictly prohibited.

APPENDIX–I

Template for the Index of Lab File

Week No.	No.	Problems with Description	Page No.	Signature of the Teacher with Date
1	1#			
	2#			
	3#			
2	1#			
	2#			
	3#			
3	1#			
	2#			
	3#			

WEEK 1

Advanced Python Data Structures and Functional Programming

Lab Description: This session delves deeper into Python's built-in data structures (lists, tuples, dictionaries, sets) and introduces more advanced manipulation techniques, including list comprehensions and basic lambda functions, setting the stage for efficient data processing.

Aim: To equip students with efficient methods for data manipulation using advanced features of Python's built-in data structures and an introduction to functional programming paradigms.

OBJECTIVES

- Master advanced list operations, including slicing with steps, and nested lists.
- Effectively use dictionary methods for data access and modification.
- Apply set operations for unique data management.
- Understand and implement list comprehensions for concise code.
- Introduce lambda functions for simple, anonymous functions.
- Utilize map and filter for common data transformations.

LEARNING OUTCOMES

Students will be able to write more compact and efficient Python code for data processing, leveraging advanced features of core data structures and functional programming constructs.

LAB ACTIVITIES

Activity 1 Scenario: Sensor Data Processing You're processing readings from a set of environmental sensors.

- Activity:
  - Given a list of sensor readings (temperatures in Celsius): readings = [22.5, 23.1, 20.9, 24.0, 21.5, 19.8, 26.2, 23.5].
  - Using list comprehension, create a new list fahrenheit_readings where each Celsius reading is converted to Fahrenheit (C×9/5+32).
  - From readings, use another list comprehension to create hot_days containing only temperatures greater than 23.0.
  - Print both new lists.

- Hint: The formula for Celsius to Fahrenheit conversion is $F = C \times 1.8 + 32$.

Activity 2 Scenario: User Preferences Management You are building a system to manage user preferences, which are stored as key-value pairs.

- Activity:
  - Create a dictionary user_preferences = {"theme": "dark", "notifications": True, "language": "en-US", "font_size": 14}.
  - Using dict.get(), safely retrieve the value for "currency" (if not present, default to "USD").
  - Update the font_size to 16.
  - Remove the "notifications" preference.
  - Print all keys and values in the updated dictionary using a loop.
- Hint: dict.get() can take a default value. Use del or pop() for removal.

Activity 3 Scenario: Unique Event IDs You have logs containing duplicate event IDs and need to find unique ones and identify common events across different logs.

- Activity:
  - log1_events = {"A101", "B202", "C303", "A101", "D404"}.
  - log2_events = {"C303", "E505", "F606", "B202"}.
  - Convert log1_events (if it were a list) into a set to find all unique events. (Assume it's initially a list ["A101", "B202", "C303", "A101", "D404"] and convert it.)
  - Find the intersection of log1_events and log2_events (events common to both).
  - Find the union of both sets (all unique events from both logs).
  - Print the results.
- Hint: Sets are inherently unique. Use set() constructor for conversion from a list.

Optional Lab Activities to be Completed at Home

Activity 4 Scenario: Data Transformation with map and filter You're applying transformations to a list of numbers.

- Activity:
  - Given numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10].
  - Use map with a lambda function to square each number in the numbers list. Convert the result to a list and print it.

- Use filter with a lambda function to get only the odd numbers from the original numbers list. Convert the result to a list and print it.

Hint: map() applies a function to all items in an input list. filter() constructs an iterator from elements for which a function returns true.

WEEK 2

Object-Oriented Programming in Python

Lab Description: This session introduces the core principles of Object-Oriented Programming (OOP) in Python. Students will learn to define classes, create objects, understand attributes and methods, and explore basic inheritance, laying a foundation for structuring complex AI/ML models.

Aim: To introduce students to OOP concepts in Python to model real-world entities and build more structured, modular, and reusable codebases.

OBJECTIVES

- Define classes and instantiate objects.
- Understand and use instance attributes and methods.
- Implement the ___init___ constructor for object initialization.
- Understand basic inheritance and method overriding.
- Differentiate between instance and class variables.
- Introduce concepts of encapsulation through private-like attributes (using _ prefix).

LEARNING OUTCOMES

Students will be able to design and implement simple classes to represent concepts, create instances of these classes, and understand how inheritance promotes code reuse.

LAB ACTIVITIES

Activity 1 Scenario: Modeling a "Car" Object You want to represent cars in a vehicle management system.

- Activity:
  - Define a Car class.
  - The Car class should have an ___init___ method that takes make, model, and year as parameters and initializes corresponding instance attributes.
  - Add a method display_info() that prints the car's make, model, and year.
  - Create two Car objects: one for a "Toyota Camry 2020" and another for a "Honda Civic 2022".

- Call display_info() for both car objects.

- Hint: Use self. to refer to instance attributes within the class.

Activity 2 Scenario: Managing a "Bank Account" You are creating a simplified bank account system.

- Activity:
  - Define a BankAccount class.
  - It should have ___init___ to set account_number and _balance (use _ to signify it's intended as a "private" attribute, though not strictly enforced in Python). Initial balance defaults to 0.
  - Add a deposit(amount) method that adds to the balance and prints the new balance.
  - Add a withdraw(amount) method that subtracts from the balance. It should check if amount is less than or equal to _balance; if not, print "Insufficient funds."
  - Create a BankAccount object. Deposit $500, then try to withdraw $200, then $400.
- Hint: Access _balance using self._balance.

Activity 3 Scenario: Building a "Student" and "GraduateStudent" Hierarchy You need to model different types of students in an academic system.

- Activity:
  - Define a Student class with ___init___ taking name and student_id. Add a get_details() method that returns a string "Name: [name], ID: [student_id]".
  - Define a GraduateStudent class that inherits from Student.
  - The GraduateStudent's ___init___ should also take thesis_topic and call the parent class's ___init___ using super().
  - Override the get_details() method in GraduateStudent to also include the thesis_topic (e.g., "Name: [name], ID: [student_id], Thesis: [thesis_topic]").
  - Create an instance of Student and an instance of GraduateStudent and call get_details() on both.
- Hint: Use super().___init___(...) in the child class constructor.

Optional Lab Activities to be Completed at Home

Activity 4 Scenario: Counting "Employees" (Class Variables) You want to keep track of the total number of employees in a company.

- Activity:

- Define an Employee class.
- Add a class variable total_employees initialized to 0.
- In the ___init___ method, increment total_employees each time a new Employee object is created.
- Create a few Employee objects.
- Print the Employee.total_employees to see the total count.

- Hint: Class variables are defined directly inside the class but outside any methods. Access them via ClassName.variable_name.

Activity 5 Scenario: Polymorphism with Shapes (Challenging Lab) You want to calculate the area of different shapes using a common interface.

- Activity:
  - Define a base class Shape with a method area() that simply raises a NotImplementedError (indicating it should be overridden by subclasses).
  - Create two subclasses: Rectangle and Circle, both inheriting from Shape.
  - Rectangle's ___init___ should take width and height. Override its area() method to return width * height.
  - Circle's ___init___ should take radius. Override its area() method to return $math.pi * radius * *2$.
  - Create instances of Rectangle and Circle. Put them in a list.
  - Iterate through the list and call the area() method on each shape, printing the result.

Hint: Import the math module for math.pi. This demonstrates polymorphism where different objects respond to the same method call in their own way.

WEEK 3

Uninformed Search: Breadth-First and Depth-First Search

Lab Description: Introduction to AI fundamentals and implementation of basic search algorithms including Breadth-First Search (BFS) and Depth-First Search (DFS).

Aim: To understand and implement fundamental search algorithms in AI.

OBJECTIVES

- Understand the concept of state space search
- Implement BFS and DFS algorithms
- Apply search algorithms to solve real-world problems

LEARNING OUTCOMES

- Students will be able to implement basic search algorithms
- Students will understand the difference between BFS and DFS
- Students will apply search algorithms to solve maze and pathfinding problems

LAB ACTIVITIES

Problem 1 Maze Solving using BFS Implement BFS to find the shortest path in a maze.

Hint: Use a queue data structure and mark visited cells to avoid cycles.

Problem 2 Family Tree Search using DFS Create a family tree and use DFS to find all descendants of a person.

Hint: Use recursion or stack to implement DFS traversal.

Problem 3 Robot in a Grid, A robot can move right or down in a grid. Some cells are blocked. Find a path from top-left to bottom-right using BFS.

Grid Example: 0010 0000 1001 0000

```
0 = free, 1 = blocked
```

Hint: Use a queue and track visited positions.

Problem 4 Website Navigation Tree, given a website's structure as a tree (or nested dictionary), simulate DFS to print the path to a given page.

Structure:

```
site = {
  'Home': {
    'About': {},
    'Products': {
      'Electronics': {},
      'Books': {}
    },
    'Contact': {}
  }
}
```

Task: Find path to 'Books' → Output: Home → Products → Books

Optional Lab Activities to be Completed at Home

Problem 5 River Crossing Problem Solve the classic farmer, fox, chicken, and grain river crossing puzzle using BFS.

Hint: Represent states as tuples and use BFS to find the solution sequence. Challenging Problem 1: Knight's Tour Find a sequence of moves for a knight to visit every square on a chessboard exactly once.

Hint: Use backtracking with DFS and implement Warnsdorff's rule for optimization.

WEEK 4

Informed Search: Best-First Search and A*

Lab Description: Implementation of informed search algorithms including Best-First Search and A* algorithm for optimal pathfinding.

Aim: To understand and implement informed search strategies using heuristics.

OBJECTIVES

- Understand the concept of heuristic functions
- Implement Best-First Search and A* algorithm
- Compare performance of informed vs uninformed search

LEARNING OUTCOMES

- Students will implement heuristic-based search algorithms
- Students will understand the importance of admissible heuristics
- Students will optimize pathfinding using informed search

LAB ACTIVITIES

Problem 1 Grid-Based Pathfinding with A*, Find the shortest path in a 2D grid using A*. Each cell is either walkable or blocked.

Hint: Use Manhattan distance as heuristic function.

Problem 2 City-to-City Travel with Best-First Search, given cities as nodes with coordinates, implement Best-First Search to find a path from one city to another. Data:

```
cities = {
   'A': (0, 0),
   'B': (2, 3),
   'C': (5, 2),
   'D': (6, 6),
   'E': (8, 3)
}
```

Heuristic: Euclidean distance to goal

Problem 3 Maze Solving with A* vs BFS, compare how A* and Breadth-First Search (BFS) perform when solving a maze. You will implement both algorithms to find a path from the start to the goal in a 2D grid-based maze. (steps taken, visited nodes, etc.). Maze Example (5×5 grid):

```
S = Start, G = Goal, 1 = Wall, 0 = Free
[
  [S, 0, 1, 0, G],
  [1, 0, 1, 0, 1],
  [0, 0, 0, 0, 0],
  [0, 1, 1, 1, 0],
  [0, 0, 0, 0, 0]
]
```

- Start: (0, 0)
- Goal: (0, 4)
- You can move in 4 directions: up, down, left, right (no diagonals)

Hint:

- Use a queue for BFS.
- Use a priority queue (heapq) for A*.
- For A*, use Manhattan distance as the heuristic:

Problem 4 Treasure Hunt on a Grid, A character must reach a treasure on a map grid with cliffs (blocked cells) and bonus cells (lower cost). Use A* to find the most efficient path.

Hint: Design an appropriate heuristic function for your problem.

Optional Lab Activities to be Completed at Home

Problem 5 Traveling Salesman Problem (TSP) Approximation Use A* with MST heuristic to approximate TSP solution. Data Source: Generate random city coordinates or use sample datasets. Challenging Problem 1: Multi-Goal Path Planning Extend A* to find optimal paths visiting multiple goals in sequence.

Hint: Use dynamic programming concepts and modify the heuristic function.

WEEK 5

Real-World Data Cleaning and Integration

Lab Description: This lab deepens the understanding of data preprocessing by applying real-world data cleaning and integration techniques. Students will tackle missing values and noisy data in a practical context and combine information from different sources using a genuine dataset.

Aim: To develop practical skills in cleaning and integrating real-world datasets, emphasizing how these steps directly impact the quality and utility of data for AI/ML models.

OBJECTIVES

- Apply strategies for handling missing data using real datasets.
- Identify and address noisy data (outliers) in a practical scenario.
- Perform data integration by merging real-world dataframes.
- Understand the challenges and nuances of real data preprocessing.

LEARNING OUTCOMES

Students will be proficient in initial data preparation steps using actual datasets, fostering a more robust understanding of data preprocessing in AI/ML.

LAB ACTIVITIES

Activity 1 Scenario: Titanic Passenger Data Cleaning You're beginning to analyze the famous Titanic dataset, but it often contains missing information. Correctly handling these missing values is critical before any predictive modeling.

- Dataset Source: Kaggle's Titanic Dataset (download train.csv): https://www.kaggle.com/c/titanic/data
- Activity:
  - Load the train.csv file into a Pandas DataFrame.
  - Identify Missing Data: Print the count of missing values for each column.
  - Strategy 1 (Removal): Drop the 'Cabin' column entirely (as it often has too many missing values to be useful without advanced imputation).
  - Strategy 2 (Imputation - Mean/Median): Fill missing values in the 'Age' column with the median age. Explain why median might be preferred over mean for 'Age'.

- Strategy 3 (Imputation - Mode): Fill missing values in the 'Embarked' column with its mode (most frequent value).
- After these steps, verify and print the remaining missing value counts for all columns.

- Hint: Use pd.read_csv(), df.isnull().sum(), df.drop(), and df.fillna(). The mode for a Series can be accessed by df['column'].mode()[0].

Activity 2 Scenario: Employee Performance Data - Outlier Detection You are evaluating employee performance data, but some entries might be erroneous or extreme outliers that could skew analysis.

- Dataset Source: (Simulated data as real performance data is sensitive, but structured like real data)
  - Create Data:

```
import pandas as pd
import numpy as np
```

```
    # Simulate performance scores (0-100) with a few outliers
np.random.seed(42) # for reproducibility
performance_scores = np.random.normal(loc=75, scale=10,
size=100)
performance_scores[10] = 150 # Extreme high outlier
performance_scores[25] = -20 # Extreme low outlier
performance_scores[50] = 5    # Less extreme low outlier
```

```
employee_df = pd.DataFrame({
    'EmployeeID': range(1, 101),
    'PerformanceScore': performance_scores.round(0)
})
print("Original Employee Performance Data (first 10
rows):\n", employee_df.head(10))
print("\nOriginal Employee Performance Data (last 10
rows):\n", employee_df.tail(10))
```

- Activity:
  - Identify Outliers (IQR Method): Calculate the first quartile (Q1) and third quartile (Q3) for 'PerformanceScore'. Then calculate the Interquartile Range (IQR = Q3 - Q1).

- Define upper bound (Q3 + 1.5 * IQR) and lower bound (Q1 - 1.5 * IQR) for outlier detection.
- Identify and print the EmployeeID and PerformanceScore of all entries that are considered outliers based on these bounds.
- Strategy (Capping/Winsorization): Replace any PerformanceScore above the upper bound with the upper bound, and any below the lower bound with the lower bound. Print the PerformanceScore column after capping, observing the change in outlier values.

- Hint: Use df.quantile(0.25) and df.quantile(0.75) for Q1 and Q3. Use boolean indexing for capping (e.g., df.loc[df['col'] > upper_bound, 'col'] = upper_bound).

Optional Lab Activities to be Completed at Home

Activity 3 Scenario: Integrating Global Temperature and City Data You want to analyze climate data and need to combine temperature records with city demographic information to see if there are correlations.

- Dataset Source:
  - Global Temperatures: (Simulated, as linking exact cities to daily temperatures globally can be complex for a basic lab. This simulates a plausible structure.)

```
import pandas as pd
np.random.seed(43)
temp_data = pd.DataFrame({
    'CityName': ['London', 'Paris', 'New York', 'Tokyo',
'Berlin', 'Rome', 'Sydney', 'Cairo'],
    'AvgTempC': np.random.uniform(10, 30, 8).round(1),
    'Humidity': np.random.uniform(50, 90, 8).round(1)
})
print("\nTemperature Data:\n", temp_data)
```

  - City Population Data: city_pop_data = pd.DataFrame({ 'City': ['London', 'Paris', 'New York', 'Tokyo', 'Sydney', 'Dubai', 'Beijing'], 'Country': ['UK', 'France', 'USA', 'Japan', 'Australia', 'UAE', 'China'], 'PopulationMillions': [9, 2.1, 8.4, 14, 5.3, 3.1, 21.5] }) print("\nCity Population Data:\n", city_pop_data)
- Activity:
  - Integrate the temp_data and city_pop_data DataFrames.
  - Perform an inner merge to keep only the cities present in both datasets. Specify the columns to merge on.
  - Print the resulting integrated DataFrame.

- Discuss: What kind of merge (left, right, outer) would you use if you wanted to keep all temperature records, even if there's no matching population data? Explain why.

- Hint: Use pd.merge(). Pay attention to the column names that match (CityName vs City) and the how='inner' parameter.

Challenging Problem 1: Complete Data Pipeline Build a comprehensive data preprocessing pipeline for a real-world dataset.

Data Source: House Prices dataset from Kaggle.

WEEK 6

Feature Selection, Normalization, and Discretization

Lab Description: This lab explores advanced data preparation techniques using real datasets. Students will practice feature selection, data normalization/standardization, and discretization, understanding their impact on model performance and interpretability.

Aim: To apply data reduction and transformation techniques to real datasets, optimizing them for various AI/ML modeling scenarios.

OBJECTIVES

- Perform feature selection on a real dataset to reduce dimensionality.
- Apply normalization and standardization to numerical features from a real dataset.
- Implement discretization (binning) on continuous features.
- Understand the practical implications of data reduction and transformation on real-world data.

LEARNING OUTCOMES

Students will be proficient in preparing datasets for machine learning by applying appropriate reduction and transformation techniques on actual data.

LAB ACTIVITIES

Activity 1 Scenario: California Housing Data - Feature Selection You are working with a housing dataset and need to select the most relevant features for predicting house prices, reducing complexity for a simpler model.

- Dataset Source: Scikit-learn's California Housing Dataset (it's built-in, no download needed, but large for actual ML):

```
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing(as_frame=True)
housing_df = housing.frame
print("California Housing Data (first 5 rows):\n",
housing_df.head())
print("\nOriginal columns:\n", housing_df.columns)
```

- Activity:
  - Feature Selection (Domain-driven): From housing_df, identify and select only the following features for your analysis: MedInc (Median Income), HouseAge, AveRooms (Average number of rooms), AveBedrms (Average number of bedrooms), and Population.
  - Create a new DataFrame selected_features_df containing only these chosen columns.
  - Print the first 5 rows of selected_features_df and its columns to confirm the reduction.
  - Discuss: Why might reducing the number of features be beneficial for a machine learning model, even if it means losing some information?
- Hint: You can select multiple columns by passing a list of column names to df[].

Activity 2 Scenario: Iris Flower Data - Normalization & Standardization The famous Iris dataset is commonly used to demonstrate classification. Scaling its features is often a good practice.

- Dataset Source: Scikit-learn's Iris Dataset (built-in):

```
    from sklearn.datasets import load_iris
iris = load_iris(as_frame=True)
iris_df = iris.frame
print("\nIris Data (first 5 rows):\n", iris_df.head())
```

- Activity:
  - Focus on the numerical features: sepal length (cm), sepal width (cm), petal length (cm), petal width (cm).
  - Normalization (Min-Max Scaling): Apply Min-Max Normalization to these four features. After scaling, each feature's values should be between 0 and 1. Print the first 5 rows of the DataFrame with normalized features.
  - Standardization (Z-score Scaling): Now, take the original four features again and apply Standardization (Z-score scaling). The formula is: X_standardized=(X–mu)/sigma. Print the first 5 rows of the DataFrame with standardized features.
  - Discuss: Explain the difference between Normalization and Standardization. When might you choose one over the other?
- Hint: For Min-Max, calculate min() and max() for each column. For Standardization, calculate mean() and std() for each column. Alternatively, use sklearn.preprocessing.MinMaxScaler and sklearn.preprocessing.StandardScaler.

Optional Lab Activities to be Completed at Home

Activity 3 Scenario: Adult Income Data - Discretization for Categorization The "Adult" dataset contains demographic information. The 'Age' feature is continuous, but for some analyses, it might be more useful to group ages into categories (bins).

- Dataset Source: UCI Adult Income Dataset (download adult.data): https://archive.ics.uci.edu/ml/datasets/Adult
  - Note: This file does not have a header. You'll need to specify column names.

```
import pandas as pd
# Download 'adult.data' to your working directory
# You might need to adjust the path
column_names = ['age', 'workclass', 'fnlwgt', 'education', 'education-
num',
                'marital-status', 'occupation', 'relationship', 'race',
'sex',
                'capital-gain', 'capital-loss', 'hours-per-week',
'native-country', 'income']
adult_df = pd.read_csv('adult.data', header=None, names=column_names,
skipinitialspace=True)
print("\nAdult Income Data (first 5 rows):\n", adult_df.head())
```

- Activity:
  - Focus on the 'age' column.
  - Discretize the 'age' column into 4 meaningful bins, creating a new column named age_group. Examples of bins: 'Young Adult' (17-30), 'Middle- Aged' (31-45), 'Senior Adult' (46-60), 'Elderly' (61+).
  - Print the unique values and their counts for the new age_group column to verify the bins.
  - Discuss: When is it beneficial to discretize a continuous variable like 'age' for a machine learning task?
- Hint: Use pd.cut(). You'll need to define the bins (cut-off points) and labels (names for the bins). For example: bins = [17, 30, 45, 60, adult_df['age'].max()], labels = ['Young Adult', 'Middle-Aged', 'Senior Adult', 'Elderly'].

Activity 4 Challenging Activity: MovieLens Dataset - Feature Engineering & Aggregation (Data Reduction/Transformation) You're working with movie rating data and want to create a reduced, more insightful dataset of movie statistics.

- Dataset Source: MovieLens 100K Dataset (download ml-100k.zip, then use u.data for ratings and u.item for movie titles): https://grouplens.org/datasets/movielens/100k/
  - Note: You'll need to extract u.data (ratings) and u.item (movie info) from the zip file.

- u.data columns: user_id | item_id | rating | timestamp (tab- separated)
- u.item columns: movie id | movie title | ... (pipe-separated)

- Activity:
  - Load u.data into ratings_df and u.item into movies_df. (You'll need to specify sep='\t' for u.data and sep='|', encoding='latin-1', header=None for u.item, taking only 0 and 1 for movie id and title).
  - Feature Engineering/Aggregation: From ratings_df, calculate the average rating and the number of ratings for each movie.
  - Data Integration & Reduction: Merge this aggregated data with the movies_df (using movie id from u.item and item_id from u.data).
  - Create a new DataFrame movie_stats_df that contains movie title, average rating, and number of ratings.
  - Sort movie_stats_df by number of ratings in descending order and print the top 10 movies.

Hint: Use `pd.read_csv()`, `df.groupby()`, `agg()`, and `pd.merge()`. A sample aggregation is shown below.

```
agg(avg_rating=('rating', 'mean'),
    num_ratings=('rating', 'count'))
```

WEEK 7

Supervised Learning: KNN and Linear Regression

Lab Description: This lab introduces students to the fundamental concepts of supervised learning, differentiating between classification (predicting categories) and regression (predicting continuous values). Students will get hands-on with K-Nearest Neighbors for classification and Linear Regression for a simple prediction task.

Aim: To introduce the core ideas of supervised learning and enable practical application of two basic algorithms: KNN for classification and Linear Regression for regression.

OBJECTIVES

- Understand the difference between classification (two-class and multiclass) and regression.
- Implement K-Nearest Neighbors (KNN) for classification.
- Implement Linear Regression for a simple regression problem.
- Calculate basic evaluation metrics for both.

LEARNING OUTCOMES

Students will be able to identify supervised learning problems, apply KNN and Linear Regression, and interpret their basic performance.

LAB ACTIVITIES

Activity 1 Scenario: Fruit Type Prediction (Multiclass Classification) You're building a system to classify fruits based on their size and color intensity.

- Activity 1.1: Data Preparation Create a dataset with two features (Size_cm, Color_Intensity) and a target FruitType (e.g., 'Apple', 'Banana', 'Orange').

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import numpy as np
```

```
data = {
    'Size_cm': [7, 7.5, 6.8, 18, 19, 17.5, 8.5, 9, 8.2, 7.2],
    'Color_Intensity': [0.8, 0.75, 0.82, 0.6, 0.58, 0.62, 0.9, 0.88, 0.92,
0.78],
    'FruitType': ['Apple', 'Apple', 'Apple', 'Banana', 'Banana', 'Banana',
'Orange', 'Orange', 'Orange', 'Apple']
}
df = pd.DataFrame(data)
X = df[['Size_cm', 'Color_Intensity']]
y = df['FruitType']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
```

```
print("Fruit Classification Data (X_train head):\n", X_train.head())
print("\nFruit Types (y_train head):\n", y_train.head())
```

- Activity 1.2: K-Nearest Neighbors (KNN) Training Train a K-Nearest Neighbors (KNN) classifier. Start with n_neighbors=3.
  - Make predictions on the X_test data.
  - Calculate and print the accuracy score of your model on the test set.
- Hint: Use KNeighborsClassifier(n_neighbors=3) and model.score().

Activity 2 Scenario: Temperature-Electricity Consumption (Simple Linear Regression) You want to predict the daily electricity consumption in a small office based on the average daily temperature.

- Activity 2.1: Data Preparation Create a simple dataset with Temperature_C and Electricity_kWh.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt
```

```
temp = np.array([10, 15, 20, 25, 30, 12, 18, 23, 28,
32]).reshape(-1, 1)
kwh = np.array([50, 60, 75, 90, 105, 55, 70, 85, 95, 110])
```

```
X_train_elec, X_test_elec, y_train_elec, y_test_elec =
train_test_split(temp, kwh, test_size=0.3, random_state=42)
```

```
print("\nElectricity Consumption Data (X_train_elec):\n",
X_train_elec[:5])
print("\nElectricity Consumption (y_train_elec):\n",
y_train_elec[:5])
```

- Activity 2.2: Linear Regression Training Train a Linear Regression model.
  - Make predictions on the test set.
  - Calculate and print the Mean Squared Error (MSE).
- Activity 2.3: Visualization Create a scatter plot of the original training data (Temperature_C vs Electricity_kWh) and draw the regression line.

Activity 1 Hint: Use LinearRegression() and plt.scatter() with plt.plot() for the regression line.

WEEK 8

Decision Trees and Logistic Regression

Lab Description: This lab expands on classification, introducing Decision Trees and Logistic Regression. Students will explore how these models create decision boundaries and understand the underlying principles like entropy and information gain (conceptually).

Aim: To deepen understanding of classification by implementing Decision Trees and Logistic Regression, and introduce the concept of decision boundaries.

OBJECTIVES

- Implement Decision Trees for classification.
- Understand the conceptual role of Entropy and Information Gain in Decision Trees.
- Implement Logistic Regression for binary classification.
- Visualize simple decision boundaries.

LEARNING OUTCOMES

Students will be able to apply Decision Trees and Logistic Regression to classification problems and grasp how these models separate data classes.

LAB ACTIVITIES

Activity 1 Scenario: Customer Behavior Classification (Two-Class with Decision Tree) You are analyzing customer data to predict if they will respond to a marketing campaign.

- Activity 1.1: Data Preparation Create a dataset with Age, PurchaseFrequency (per month), and RespondedToCampaign (0 for No, 1 for Yes).

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np
```

```
data = {
    'Age': np.random.randint(20, 60, 100),
    'PurchaseFrequency': np.random.uniform(0.5, 5, 100).round(1),
    'RespondedToCampaign': np.random.choice([0, 1], 100, p=[0.6, 0.4])
}
# Make response somewhat dependent on younger age and higher frequency
for i in range(100):
    if data['Age'][i] < 35 and data['PurchaseFrequency'][i] > 3:
        data['RespondedToCampaign'][i] = 1
    elif data['Age'][i] > 50 and data['PurchaseFrequency'][i] < 1.5:
        data['RespondedToCampaign'][i] = 0
```

```
df = pd.DataFrame(data)
X = df[['Age', 'PurchaseFrequency']]
y = df['RespondedToCampaign']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
```

```
print("Campaign Response Data (X_train head):\n", X_train.head())
print("\nCampaign Response Target (y_train head):\n", y_train.head())
```

- Activity 1.2: Decision Tree Training Train a Decision Tree Classifier. Set max_depth=3 to keep the tree simple and interpretable.
  - Calculate and print its accuracy score.
- Activity 1.3: Conceptual Understanding of Tree Splitting Explain, in simple terms, how a Decision Tree ”decides” which feature to split on at each step. (Hint: Think about making the groups as ”pure” as possible after a split).

Activity 2 Scenario: Email Spam Detection (Two-Class with Logistic Regression) You’re building a basic system to identify if an email is spam (1) or not spam (0) based on features like the number of suspicious keywords.

- Activity 2.1: Data Preparation Create a dataset with NumSuspiciousKeywords and IsSpam.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
```

```
np.random.seed(42)
keywords = np.random.randint(0, 10, 100) # Number of suspicious keywords
# Higher keywords = higher spam probability
spam_prob = 1 / (1 + np.exp(-(keywords - 5))) # Sigmoid-like function
is_spam = (np.random.rand(100) < spam_prob).astype(int)
```

```
df = pd.DataFrame({'NumSuspiciousKeywords': keywords, 'IsSpam': is_spam})
X = df[['NumSuspiciousKeywords']] # Features must be 2D
y = df['IsSpam']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
```

```
print("\nSpam Detection Data (X_train head):\n", X_train.head())
print("\nSpam Target (y_train head):\n", y_train.head())
```

- Activity 2.2: Logistic Regression Training Train a Logistic Regression model.
  - Calculate and print its accuracy score.

Optional Lab Activities to be Completed at Home

Activity 1 Activity 2.3: Conceptual Decision Boundary For this 1-feature problem, where would the Logistic Regression model likely place its decision boundary to classify an email as spam or not spam? (Think about a threshold value for NumSuspiciousKeywords).

WEEK 9

Artificial Neural Networks: Perceptron and MLP

Lab Description: This lab introduces the foundational concepts of Artificial Neural Networks (ANNs), starting with the basic Perceptron and moving to the Multilayer Perceptron (MLP). Students will implement a simplified Perceptron and use a high-level library for an MLP.

Aim: To provide an initial hands-on and conceptual understanding of neural network architecture, activation functions, and the basic learning process.

OBJECTIVES

- Understand the concept of a Perceptron as a basic neural unit.
- Implement a simplified Perceptron for a basic logic gate.
- Understand the architecture of a Multilayer Perceptron (MLP).
- Train an MLP for classification using scikit-learn.
- Grasp the conceptual idea of the Backpropagation algorithm.

LEARNING OUTCOMES

Students will be able to build and train simple neural networks and differentiate between single-layer and multi-layer architectures.

LAB ACTIVITIES

Activity 1 Scenario: Perceptron for Basic Logic (AND Gate) You'll implement a Perceptron to mimic an AND logic gate.

- Activity 1.1: Data Preparation Define inputs and target outputs for an AND gate:

```
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
# AND gate inputs and outputs
```

```
X_and = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_and = np.array([0, 0, 0, 1]) # AND gate outputs
```

- Activity 1.2: Simplified Perceptron Implementation
  - Implement a simple Perceptron:
  - Initialize random weights (e.g., np.random.rand(2) * 0.1) and a random bias (e.g., np.random.rand(1)[0] * 0.1).
  - Define a step_function (activation function): 1 if z >= 0, else 0.
  - Set learning_rate = 0.1 and epochs = 20.
  - Loop through epochs. In each epoch, iterate through each (x, y_true) pair from X_and, y_and:
  - Calculate z = np.dot(x, weights) + bias.
  - Get prediction = step_function(z).
  - Calculate error = y_true - prediction.
  - Update weights = weights + learning_rate * error * x.
  - Update bias = bias + learning_rate * error.
  - After training, test your Perceptron by making predictions for all four AND gate inputs (X_and) and print the results.
- Hint: Focus on implementing the weight and bias update rule correctly.

Activity 2 Scenario: Multilayer Perceptron for Classification You'll use an MLP to classify basic shapes (e.g., squares vs. circles) represented by simplified features.

- Activity 2.1: Data Preparation Create a dataset with features like NumCorners, AspectRatio and target ShapeType (0 for square, 1 for circle).

```
import pandas as pd
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
# Sample data for shapes
# Features: NumCorners, AspectRatio
# Target: 0 = Square, 1 = Circle
```

```
X_shapes = np.array([
    [4, 1.0], [4, 1.05], [4, 0.95], # Squares
    [0, 1.0], [0, 1.1], [0, 0.9],   # Circles
    [4, 0.5], [0, 2.0]              # Some noisy or mislabeled
points
])
y_shapes = np.array([0, 0, 0, 1, 1, 1, 0, 1]) # Corresponding
labels
```

```
X_train_s, X_test_s, y_train_s, y_test_s =
train_test_split(X_shapes, y_shapes, test_size=0.3,
random_state=42)
```

```
print("Shape Classification Data (X_train_s):\n", X_train_s)
print("\nShape Target (y_train_s):\n", y_train_s)
```

- Activity 2.2: Training an MLPClassifier
  - Initialize an MLPClassifier. Set hidden_layer_sizes=(5,) (one hidden layer with 5 neurons), max_iter=500, random_state=42, and solver='adam'.
  - Train the MLP classifier on X_train_s and y_train_s.
  - Make predictions on X_test_s.
  - Calculate and print the accuracy score.

Optional Lab Activities to be Completed at Home

- Activity 2.3: Conceptual Backpropagation Briefly explain, in your own words, what happens during the Backpropagation algorithm in an MLP. How does it help the network learn? (Focus on the idea of error going backward to adjust weights).

Activity 3 Problem 3: Image Classification with Neural Networks Apply neural networks to classify handwritten digits. Data Source: MNIST dataset or Fashion-MNIST dataset

Activity 4 Problem 4: Activation Functions Comparison Compare different activation functions (sigmoid, tanh, ReLU).

Hint: Analyze convergence speed and performance on same dataset.

WEEK 10

Unsupervised Learning: K-Means and Hierarchical Clustering

Lab Description: This lab introduces students to unsupervised learning, where models discover patterns and structures in unlabeled data. Students will apply K-Means and Hierarchical Clustering to group data points based on similarity.

Aim: To enable students to understand and implement basic clustering techniques for grouping similar data points without prior knowledge of categories.

OBJECTIVES

- Differentiate between supervised and unsupervised learning.
- Implement K-means clustering for grouping data.
- Understand the concept of centroids and elbow method conceptually.
- Implement Hierarchical Clustering and interpret a dendrogram.

LEARNING OUTCOMES

Students will be able to apply K-Means and Hierarchical Clustering to simple datasets and interpret their results, understanding when to use unsupervised learning.

LAB ACTIVITIES

Activity 1 Scenario: Animal Grouping (K-Means Clustering) You have data on various animals based on their weight and height, and you want to see if they naturally form groups (e.g., small, medium, large animals).

- Activity 1.1: Data Preparation Create a dataset with Weight_kg and Height_cm for different animals.

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```
np.random.seed(42)
```

```
# Create data for 3 conceptual groups: Small, Medium, Large
small_animals = np.random.normal(loc=[5, 20], scale=[1, 5],
size=(20, 2))
medium_animals = np.random.normal(loc=[50, 80], scale=[10, 15],
size=(20, 2))
large_animals = np.random.normal(loc=[300, 150], scale=[50, 25],
size=(20, 2))
```

```
X_animals = pd.DataFrame(np.vstack([small_animals,
medium_animals, large_animals]), columns=['Weight_kg',
'Height_cm'])
```

```
# Scale the data (important for K-Means)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_animals)
X_scaled_df = pd.DataFrame(X_scaled, columns=X_animals.columns)
```

```
print("Scaled Animal Data (first 5 rows):\n", X_scaled_df.head())
```

- Activity 1.2: K-Means Clustering Training Apply K-Means clustering to X_scaled_df. Set n_clusters=3.
  - Fit the K-Means model to the scaled data.
  - Get the cluster labels for each data point and add them as a new column Cluster to X_animals (the original unscaled DataFrame).
- Activity 1.3: Visualization and Discussion Create a scatter plot of Weight_kg vs Height_cm from X_animals, coloring data points by their assigned Cluster. Plot the cluster centroids as well.
  - Discuss: In your own words, what does a centroid represent in K-Means clustering?

Activity 2 Scenario: Grouping Students by Study Habits (Hierarchical Clustering) You have data on students' weekly study hours and average sleep hours, and you want to find natural groupings of study habits.

- Activity 2.1: Data Preparation Create a dataset with Study_Hours_Week and Sleep_Hours_Night.

```
import pandas as pd
import numpy as np
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt
```

```
np.random.seed(42)
```

```
# Create synthetic data with some groupings
X_students = pd.DataFrame({
    'Study_Hours_Week': np.random.randint(5, 40, 50),
    'Sleep_Hours_Night': np.random.uniform(4, 9, 50).round(1)
})
```

```
# Add some patterns:
X_students.loc[X_students['Study_Hours_Week'] > 30,
'Sleep_Hours_Night'] = np.random.uniform(4, 6,
sum(X_students['Study_Hours_Week'] > 30)).round(1) # High study,
low sleep
X_students.loc[X_students['Study_Hours_Week'] < 10,
'Sleep_Hours_Night'] = np.random.uniform(7, 9,
sum(X_students['Study_Hours_Week'] < 10)).round(1) # Low study,
high sleep
```

```
print("\nStudent Habit Data (first 5 rows):\n",
X_students.head())
```

- Activity 2.2: Hierarchical Clustering Perform hierarchical clustering on X_students using the linkage function from scipy.cluster.hierarchy. Use the 'ward' method.

Optional Lab Activities to be Completed at Home

- Activity 2.3: Dendrogram Interpretation Generate and plot a dendrogram from the linkage results.

Discuss: Based on the dendrogram, if you were to divide students into 2 main groups, where would you "cut" the dendrogram? What would these two groups conceptually represent (e.g., "high study" vs. "low study")?

WEEK 11

Underfitting, Overfitting, and Data Separability

Lab Description: This lab introduces the critical concepts of underfitting and overfitting in machine learning models. Students will also explore the distinction between linearly separable and non-linearly separable datasets, understanding how this impacts model choice.

Aim: To help students identify and understand underfitting, overfitting, and the characteristics of linearly and non-linearly separable data.

OBJECTIVES

- Define and illustrate underfitting and overfitting.
- Differentiate between linearly separable and non-linearly separable datasets.
- Observe model behavior on different types of data.

LEARNING OUTCOMES

Students will be able to recognize underfit and overfit models and identify the separability of a dataset.

LAB ACTIVITIES

Activity 1 Scenario: Simple Regression - Underfitting vs. Overfitting You're trying to fit a curve to data points. A very simple model might underfit, while a very complex one might overfit.

- Activity 1.1: Data Preparation Generate a synthetic dataset with a non-linear relationship (e.g., a noisy sine wave).

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error
```

```
np.random.seed(0)
X = np.sort(5 * np.random.rand(80, 1), axis=0)
```

```
y = np.sin(X).ravel() + np.random.normal(0, 0.5, 80) # Noisy sine
wave
```

```
# Split into training and testing
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

```
plt.figure(figsize=(8, 5))
plt.scatter(X_train, y_train, label='Training Data')
plt.scatter(X_test, y_test, label='Testing Data', alpha=0.6)
plt.title("Original Data")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

- Activity 1.2: Underfitting a Linear Model Train a simple Linear Regression model on X_train, y_train.
  - Make predictions on X_test.
  - Plot the original data points and the linear regression line.
  - Discuss: Does the straight line capture the pattern well? Why is this an example of underfitting?
- Activity 1.3: Overfitting a High-Degree Polynomial Model Train a Polynomial Regression model with a high degree (e.g., degree=20) on X_train, y_train.
  - Make predictions on X_test.
  - Plot the original data points and the high-degree polynomial curve.
  - Discuss: How well does the curve fit the training data? How well does it fit the testing data? Why is this an example of overfitting?

Activity 2 Scenario: Data Separability - Linear vs. Non-linear Boundaries Some classification problems can be solved with a straight line, while others need more complex curves.

- Activity 2.1: Linearly Separable Data Generate two classes of data that can be perfectly separated by a straight line.

```
from sklearn.datasets import make_classification
from sklearn.svm import SVC
```

```
X_linear, y_linear = make_classification(n_samples=100,
n_features=2, n_redundant=0, n_informative=2,
                                        n_clusters_per_class=1,
random_state=4, class_sep=1.5) # High class_sep for linear
separability
```

```
plt.figure(figsize=(8, 5))
plt.scatter(X_linear[:, 0], X_linear[:, 1], c=y_linear,
cmap='viridis', s=50)
plt.title("Linearly Separable Data")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

- Train a Logistic Regression model on X_linear, y_linear.
- Discuss: Based on the plot and the model type, is this dataset linearly separable? How can you tell?

Optional Lab Activities to be Completed at Home

- Activity 2.2: Non-linearly Separable Data Generate two classes of data that cannot be separated by a straight line (e.g., concentric circles or XOR-like pattern).

```
from sklearn.datasets import make_circles
X_nonlinear, y_nonlinear = make_circles(n_samples=100, noise=0.1,
factor=0.5, random_state=1)
```

```
 plt.figure(figsize=(8, 5))
plt.scatter(X_nonlinear[:, 0], X_nonlinear[:, 1], c=y_nonlinear,
cmap='plasma', s=50)
plt.title("Non-linearly Separable Data")
```

```
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

- Train a Logistic Regression model on X_nonlinear, y_nonlinear.
- Discuss: Can a straight line effectively separate these two classes? Why is this dataset non-linearly separable?

WEEK 12

Data Splitting and K-Fold Cross-Validation

Lab Description: This lab focuses on crucial techniques for preparing data for robust model training and evaluation. Students will learn how to split data into training, testing, and validation sets, and implement K-fold cross-validation to get a more reliable estimate of model performance.

Aim: To teach students best practices for dividing datasets to avoid bias in model evaluation and to improve the reliability of performance estimates.

OBJECTIVES

- Understand the purpose of training, testing, and validation sets.
- Implement manual splitting of data into these sets.
- Implement K-fold cross-validation using scikit-learn.
- Explain the advantages of cross-validation over a single train-test split.

LEARNING OUTCOMES

Students will be able to correctly partition data for machine learning tasks and apply K-fold cross-validation.

LAB ACTIVITIES

Activity 1 Scenario: Customer Churn Data - Training, Testing & Validation Sets You're analyzing customer churn data and need to ensure your model is evaluated fairly on unseen data.

- Activity 1.1: Data Preparation Load a dataset (e.g., a simplified simulated churn dataset).

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import numpy as np
```

```
# Simulated Churn Data
```

```
np.random.seed(0)
churn_data = pd.DataFrame({
    'MonthlyCharges': np.random.uniform(20, 100, 200),
    'DataUsageGB': np.random.uniform(10, 80, 200),
    'ContractYears': np.random.randint(1, 4, 200),
    'Churn': np.random.choice([0, 1], 200, p=[0.7, 0.3])
})
X = churn_data.drop('Churn', axis=1)
y = churn_data['Churn']
```

```
print("Original Churn Data (first 5 rows):\n", churn_data.head())
```

- Activity 1.2: Train-Test Split Split X and y into training (70%) and testing (30%) sets.
  - Print the shapes of the resulting training and testing sets.
- Activity 1.3: Train-Validation-Test Split Now, split the data into training (60%), validation (20%), and testing (20%) sets. (Hint: Perform two train_test_split calls).
  - Print the shapes of all three resulting sets.
  - Discuss: Why do we separate data into training, testing, and sometimes a validation set? What is the specific purpose of the validation set?

Activity 2 Scenario: K-Fold Cross-Validation for Robust Evaluation You want a more reliable estimate of how your model will perform on new, unseen data, rather than relying on a single train-test split.

- Activity 2.1: K-Fold Implementation Use the churn_data from Activity 1.1.
  - Apply K-fold cross-validation (e.g., KFold(n_splits=5)) with a Logistic Regression model.
  - For each fold, train the model and calculate its accuracy.
  - Store the accuracy for each fold.
  - Calculate and print the average accuracy across all folds.
- Hint: Use from sklearn.model_selection import KFold, cross_val_score.

Optional Lab Activities to be Completed at Home

- Activity 2.2: Discussion What are the advantages of using K-fold cross- validation compared to a single train-test split for evaluating a model?

WEEK 13

Classification Accuracy and Confusion Matrix

Lab Description: This lab introduces fundamental performance metrics for classification tasks. Students will learn to calculate and interpret overall accuracy and dive deeper into the confusion matrix to understand True Positives, True Negatives, False Positives, and False Negatives.

Aim: To equip students with the ability to evaluate classification models effectively using accuracy and the confusion matrix.

OBJECTIVES

- Calculate classification accuracy.
- Construct and interpret a confusion matrix.
- Understand the meaning of True Positives, True Negatives, False Positives, and False Negatives.

LEARNING OUTCOMES

Students will be able to use key metrics to assess how well a classification model performs.

LAB ACTIVITIES

Activity 1 Scenario: Email Spam Detection - Understanding Correct/Incorrect Predictions You have built a simple spam detector and want to see how well it classifies emails.

- Activity 1.1: Data Preparation Simulate actual (true) labels and predicted labels for a binary classification problem (e.g., Spam/Not Spam).

```
import numpy as np
from sklearn.metrics import accuracy_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# True labels (0 = Not Spam, 1 = Spam)
y_true_spam = np.array([0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1,
1, 0, 1, 0, 1, 0])
# Predicted labels by your model
```

```
y_pred_spam = np.array([0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1,
0, 0, 1, 0, 1, 0])
```

```
print("True Spam Labels:\n", y_true_spam)
print("Predicted Spam Labels:\n", y_pred_spam)
```

- Activity 1.2: Calculate Accuracy Calculate and print the classification accuracy of the spam detector.

Optional Lab Activities to be Completed at Home

- Activity 1.3: Confusion Matrix Construction Construct the confusion matrix for these predictions.
  - Print the confusion matrix.
  - Identify: From the confusion matrix, manually identify and print the values for:
  - True Positives (TP): Correctly predicted as Spam (actual Spam).
  - True Negatives (TN): Correctly predicted as Not Spam (actual Not Spam).
  - False Positives (FP): Predicted as Spam, but actually Not Spam (Type I error).
  - False Negatives (FN): Predicted as Not Spam, but actually Spam (Type II error).
  - Discuss: Why is looking at just accuracy sometimes not enough for model evaluation, especially in imbalanced datasets (e.g., very few spam emails)?

WEEK 14

Precision, Recall, and Regression Error Measures

Lab Description: This lab extends performance evaluation to more nuanced metrics: precision and recall. Students will understand when to prioritize one over the other and will conceptually grasp different error measures beyond simple accuracy.

Aim: To enable students to apply and interpret precision and recall, and to understand the broader concept of error measures in machine learning.

OBJECTIVES

- Calculate and interpret Precision.
- Calculate and interpret Recall.
- Understand the trade-off between precision and recall.
- Conceptually understand various error measures (e.g., for regression tasks).

LEARNING OUTCOMES

Students will be able to choose and apply appropriate evaluation metrics based on the specific problem context.

LAB ACTIVITIES

Activity 1 Scenario: Disease Diagnosis - Prioritizing Recall or Precision You have a model predicting if a patient has a rare disease (1) or not (0). Missing a true positive (False Negative) is very costly.

- Activity 1.1: Data Preparation Use the same y_true_spam and y_pred_spam from Lab 3, but now interpret 1 as 'Disease Present' and 0 as 'No Disease'.

```
import numpy as np
from sklearn.metrics import recall_score, precision_score
```

```
# Same data as Lab 3, but re-contextualized
y_true_disease = np.array([0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0,
```

```
1, 1, 0, 1, 0, 1, 0])
y_pred_disease = np.array([0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0,
1, 0, 0, 1, 0, 1, 0])
```

```
# For clarity:
# TP = 7 (correctly identified disease)
# TN = 9 (correctly identified no disease)
# FP = 1 (predicted disease, but no disease)
# FN = 3 (predicted no disease, but actually disease)
```

- Activity 1.2: Calculate Precision and Recall Calculate and print the Precision score. Calculate and print the Recall score.
- Activity 1.3: Discussion on Trade-off In this medical diagnosis scenario, would you prioritize high recall or high precision? Why? (Think about the consequences of False Positives vs. False Negatives).

Activity 2 Scenario: Regression Error Measures - Beyond MSE You are predicting numerical values (e.g., house prices) and want to understand different ways to quantify prediction errors.

- Activity 2.1: Data Preparation Simulate true house prices and your model's predicted prices.

```
from sklearn.metrics import mean_absolute_error, r2_score
```

```
# True House Prices (in thousands USD)
y_true_prices = np.array([200, 250, 300, 350, 400, 220, 280, 330,
380, 420])
# Predicted House Prices
y_pred_prices = np.array([210, 240, 310, 360, 390, 230, 270, 340,
370, 430])
```

```
print("True Prices:\n", y_true_prices)
print("Predicted Prices:\n", y_pred_prices)
```

Optional Lab Activities to be Completed at Home

- Activity 2.2: Calculate Error Measures Calculate and print the following:
  - Mean Absolute Error (MAE): The average absolute difference between true and predicted values.
  - Root Mean Squared Error (RMSE): (Calculate MSE first, then take square root). This penalizes larger errors more.
  - R-squared (R2 Score): How well future samples are likely to be predicted by the model.
- Hint: Use mean_absolute_error, mean_squared_error (then np.sqrt()), and r2_score.
- Activity 2.3: Discussion on Error Interpretation If you saw a large RMSE but a small MAE, what would that tell you about the type of errors your model is making? (Hint: Think about which metric is more sensitive to large errors).